

CLAUDE CODE DEEP DIVE

Basic Todo App — Vite + React + TypeScript + Express + TanStack Query

Source: README.md

A minimal full-stack todo application that demonstrates how to wire a **Vite + React + TypeScript** frontend to an **Express + TypeScript** backend, using **TanStack Query (v5)** for data fetching, caching, and mutations.

Todos are kept in an in-memory array on the server (no database), so the list resets when the server restarts. Perfect as a learning sandbox.

Tech Stack

Layer	Technology
Frontend	Vite, React 19, TypeScript, TanStack Query v5
Backend	Node.js, Express, TypeScript, tsx
Storage	In-memory array (resets on server restart)
Styling	Plain CSS

■ Project Structure

```

6.Tutorial5/
├── README.md
├── Notes.md
├── client/                                # Vite + React + TS frontend
│   ├── src/
│   │   ├── main.tsx                    # Wraps App in QueryClientProvider
│   │   ├── App.tsx                     # Todo UI: useQuery + useMutation
│   │   ├── api.ts                      # Typed fetch helpers
│   │   └── App.css                     # Styles
│   ├── package.json
│   └── tsconfig.json
├── server/                              # Express + TS backend
│   ├── src/
│   │   ├── index.ts                   # Express app + routes
│   │   ├── store.ts                  # In-memory todo store
│   │   └── types.ts                   # Todo type
│   ├── package.json
│   └── tsconfig.json

```

■ Prerequisites

You need these installed on your machine:

- **Node.js** ≥ 20 (recommended 22). Verify with `node -v`.
- **npm** ≥ 10 . Verify with `npm -v`.
- **git** (only if you are cloning the repo).

If you don't have Node, install it from <https://nodejs.org/> or via a version manager such as `nvm` / `fnm` / `volta`.

■ Installation

After cloning or copying the project, install dependencies in **both** subprojects.

BASH

```
# from the project root (6.Tutorial5/)  
  
# 1. Install server dependencies  
cd server  
npm install  
cd ..  
  
# 2. Install client dependencies  
cd client  
npm install  
cd ..
```

That's it — no global tools needed. `tsx` (server dev runner) and `vite` (client dev runner) are local devDependencies.

■ Running the App in Development

You need **two terminals** open at the same time.

Terminal 1 — start the backend

BASH

```
cd server  
npm run dev
```

You should see:

```
Server listening on http://localhost:3001
```

The server uses `tsx watch`, so any change to `server/src/*.ts` triggers a restart.

Terminal 2 — start the frontend

BASH

```
cd client  
npm run dev
```

Vite will print:

```
VITE v8.x.x ready in xxx ms  
  
→ Local: http://localhost:5173/
```

Open <http://localhost:5173> in a browser. You can now add, toggle, and delete todos — every action calls the backend through TanStack Query and the list refreshes via cache invalidation.

■ API Endpoints

The Express backend exposes a small REST API on `http://localhost:3001`.

Method	Path	Body	Returns	Notes
GET	<code>/todos</code>	—	<code>Todo[]</code>	List all todos
POST	<code>/todos</code>	<code>{ "title": "..."} </code>	<code>Todo</code> (201)	Creates a new todo
PATCH	<code>/todos/:id</code>	<code>{ "title"?: string, "completed"?: bool }</code>	<code>Todo</code>	Updates a field; 404 if missing
DELETE	<code>/todos/:id</code>	—	empty body (204)	Deletes; 404 if missing

`Todo` shape:

```
{
  id: string;           // uuid
  title: string;
  completed: boolean;
}
```

TS

Quick API smoke test (with curl)

```
# list (empty initially)
curl http://localhost:3001/todos

# create
curl -X POST http://localhost:3001/todos \
  -H 'Content-Type: application/json' \
  -d '{"title":"buy milk"}'

# toggle (replace <id> with the returned id)
curl -X PATCH http://localhost:3001/todos/<id> \
  -H 'Content-Type: application/json' \
  -d '{"completed":true}'

# delete
curl -X DELETE http://localhost:3001/todos/<id>
```

BASH

■ Available Scripts

Server (`cd server`)

Command	What it does
<code>npm run dev</code>	Start dev server with auto-reload (<code>tsx watch</code>)
<code>npm run build</code>	Compile TypeScript to <code>dist/</code>
<code>npm start</code>	Run the compiled server (<code>node dist/index.js</code>)
<code>npm run typecheck</code>	Type-check without emitting (<code>tsc --noEmit</code>)

Client (`cd client`)

Command	What it does
<code>npm run dev</code>	Start Vite dev server with HMR
<code>npm run build</code>	Type-check (<code>tsc -b</code>) and produce a production build in <code>dist/</code>
<code>npm run preview</code>	Serve the production build locally
<code>npm run lint</code>	Run ESLint

■ Production Build

BASH

```
# Build the server
cd server
npm run build
npm start           # serves on :3001

# Build the client (in another terminal)
cd client
npm run build
npm run preview     # serves the static build
```

For a real deployment you would host `client/dist/` behind a static file server (e.g. nginx, Vercel, Netlify) and run the compiled server (`node server/dist/index.js`) on a Node host. Make sure the client's `API` constant in `client/src/api.ts` points to the deployed backend URL.

■ How the Pieces Fit Together

1. `client/src/main.tsx` creates one `QueryClient` and wraps `<App />` in `<QueryClientProvider>`. Every component below can now read and mutate the cache.
2. `client/src/api.ts` exports typed `fetch`-based helpers (`fetchTodos` , `createTodo` , `updateTodo` , `deleteTodo`). They throw on non-OK responses so TanStack Query can surface errors.
3. `client/src/App.tsx` uses:
 - `useQuery({ queryKey: ['todos'], queryFn: fetchTodos })` to load the list. The v5 `isPending` flag drives the loading UI.
 - Three `useMutation` hooks (add / toggle / delete) — each calls `queryClient.invalidateQueries({ queryKey: ['todos'] })` on success so the list automatically refetches.
4. `server/src/index.ts` wires `cors()` + `express.json()` and registers the four CRUD routes against the in-memory `store`.
5. `server/src/store.ts` holds a module-level `todos: Todo[]` array and exposes `list / create / update / remove` helpers; ids come from `crypto.randomUUID()`.

■ Troubleshooting

- **Port 3001 already in use** — another process is on that port. Find it (`lsof -i :3001`) and stop it, or change the `PORT` constant in `server/src/index.ts`. If you change the backend port, also update `API` in `client/src/api.ts`.
- **Port 5173 already in use** — Vite will offer to start on the next free port. You can also pass `npm run dev -- --port 5174`.
- **CORS error in the browser console** — make sure the backend is running and that `app.use(cors())` is enabled in `server/src/index.ts` (it is by default).
- **Todos disappear after server restart** — expected. Storage is in-memory by design; switch to JSON-on-disk or SQLite if you want persistence.
- **TypeScript errors on install** — run `npm run typecheck` in `server/` and `npx tsc -b --noEmit` in `client/` to see exactly what's failing.